

PROJECT DOCUMENTATION

AI-Career Companion Agent for Internship Matching & Interview Preparation

An end-to-end, AI-powered career development platform featuring resume parsing, semantic internship matching, cover letter generation, a RAG career advisor, an interview preparation agent, and a multilingual voice resume architect.

Project Title: AI-Career Companion Agent for Internship Matching & Interview Preparation

Prepared By: Swaroop Ranjan Rout

Batch : 02

Team Code: #M-2-3

Mentor: Vijay Parmar

Table of Contents

- 1. Project Overview 4**
 - 1.1 Project Name4
 - 1.2 What is the Project?.....4
 - 1.3 Problem Statement.....4
 - 1.4 Target Users.....4
- 2. Project Objective 4**
 - Objectives4
- 3. Technology Stack..... 5**
 - Backend5
 - Frontend.....6
- 4. Project Requirements..... 7**
 - Functional Requirements7
 - Non-Functional Requirements.....7
- 5. System Architecture / Workflow 7**
 - Architecture Overview7
 - Request Flow Examples7
 - High-Level Data Flow8
- 6. Backend Service Breakdown 9**
 - 6.1 Authentication & User Session Management9
 - 6.2 Resume Ingestion & Extraction Engine.....9
 - 6.3 Multi-Stage Semantic Internship Matcher9
 - 6.4 AI Cover Letter Generator.....9
 - 6.5 RAG Career Knowledge Assistant9
 - 6.6 Interview Preparation & Mock Interview Agent10
 - 6.7 Voice Resume Architect & Multilingual Translation10
- 7. Database Schema & Entity Models..... 11**
 - Table: users.....11
 - Table: blacklisted_tokens11
 - Table: resumes.....11
 - Table: cover_letters12
- 8. Complete REST API Specification 13**
 - Authentication (/auth).....13
 - Resume & Matching (/resume).....13
 - Cover Letter (/cover-letter).....13
 - Career Assistant RAG (/career-assistant)13
 - Interview Agent (/interview-agent)13
 - Voice Resume Architect (/voice-resume).....13
- 9. Frontend Architecture & UI Systems 14**
 - App Layout & Routing.....14
 - Client Architecture Patterns.....14
- 10. Environment Configuration (.env)..... 14**
- 11. Step-by-Step Setup Guide on a New Machine 15**

Step 1: Clone or Copy Repository	15
Step 2: Set Up Backend Environment (Python 3.10+)	15
Step 3: Configure Environment Variables.....	15
Step 4: Build RAG Index.....	15
Step 5: Initialize Frontend Environment (Node.js 18+).....	15
Step 6: Start Servers	15
12. Deployment Guide (Railway + Vercel).....	15
Backend → Railway	15
Frontend → Vercel	16
13. Testing	17
Testing Types Performed.....	17
Test Cases	17
14. Challenges & Solutions	17
15. Final Project Status	17
Completed Features	17
Final Outcome	17
Appendix: Summary for LLM Agents & Re-implementers.....	18

1. Project Overview

1.1 Project Name

AI-Career Companion Agent for Internship Matching & Interview Preparation

1.2 What is the Project?

The AI-Career Companion Agent is a unified, AI-powered career development platform for students and early-career job seekers. It combines automated resume parsing, multi-stage semantic internship matching, AI-generated cover letters, a grounded RAG-based career advisor, an interactive multi-turn interview preparation agent, and a multilingual voice-first resume architect with executive ATS PDF export. The platform allows a user to go from raw resume (or even a spoken conversation) to a matched internship, a tailored cover letter, and a fully rehearsed mock interview — all within one application.

1.3 Problem Statement

Modern career-readiness platforms suffer from high barriers to entry: complex resume builders, rigid upload requirements, language barriers for non-native English speakers, and a lack of personalized, actionable interview guidance. Candidates often struggle to identify internships that truly match their skills, to write role-specific cover letters, and to prepare effectively for interviews without expensive coaching. This project addresses these gaps with an integrated, AI-driven solution.

1.4 Target Users

The application is designed for the following user groups:

- Students seeking internships
- Early-career job seekers
- Candidates who prefer speaking over typing (multilingual voice input)
- Career mentors / placement cells reviewing candidate readiness

2. Project Objective

The project aims to deliver an end-to-end AI career companion that removes friction from the internship search and interview preparation process.

Objectives:

- Automate resume parsing and structured candidate data extraction from PDF/DOCX files.
- Provide multi-stage, semantically accurate internship matching using dense vector retrieval, feature-based reranking, and LLM-based match explanations.
- Generate tailored, role-specific cover letters aligned to a candidate's resume and the target job description.
- Offer a grounded RAG career advisor that answers questions using a curated knowledge base with source citations.
- Provide an interactive interview preparation agent with role recommendations, STAR-based HR questions, project-defense questions, personalized roadmaps, and multi-turn mock interviews with rubric scoring.
- Enable a voice-first, multilingual resume builder (Telugu, Hindi, Tamil, English, etc.) that translates, cleans, and structures spoken input into an ATS-compliant resume with one-click PDF export.

3. Technology Stack

Backend

Technology	Version	Purpose
Python	3.10+	Core language runtime
FastAPI	^0.110.0	High-performance async web framework
Uvicorn	^0.28.0	ASGI server
SQLAlchemy	^2.0.0	Relational ORM (SQLite / PostgreSQL)
Pydantic	^2.6.0	Data validation and schema serialization
python-jose[cryptography]	^3.3.0	JWT token encoding, decoding, and verification
passlib[bcrypt] & bcrypt	^1.7.4 / 4.0.1	Password hashing (Bcrypt algorithm)
Groq SDK	^0.9.0	Ultra-fast LLM inference API (gpt-oss-120b & llama-3.3-70b)
Sentence-Transformers	^2.2.0	Dense vector embeddings (all-MiniLM-L6-v2)
FAISS-CPU	^1.7.4	Efficient vector similarity search
pdfplumber & PyPDF2	^0.11.0	PDF text and table extraction
python-docx	^1.1.0	DOCX Word document extraction
python-multipart	^0.0.9	Form data and multi-part file uploads
python-dotenv	^1.0.1	Environment variable management

Frontend

Technology	Version	Purpose
Node.js	18+ or 20+	JavaScript runtime environment
React	^19.2.8	Component-based view library
TypeScript	~6.0.2	Static type safety and developer productivity
Vite	^8.2.0	Frontend build tool and dev server
React Router	^7.18.2	Declarative routing with protected route layout
Tailwind CSS	^4.3.3	Utility styling and responsive design tokens
jsPDF	^4.2.1	Client-side ATS resume PDF generation
Web Speech API	Native	In-browser speech recognition and text-to-speech

4. Project Requirements

Functional Requirements

- User should be able to register/login securely using JWT-based authentication.
- User should be able to upload a resume (PDF/DOCX) for parsing.
- System should extract structured candidate data (contact info, skills, experience, education, projects, certifications).
- System should recommend matching internships based on the parsed resume.
- User should be able to generate a tailored cover letter for a specific company/role.
- User should be able to ask career-related questions and receive grounded, cited answers.
- User should be able to run an interactive mock interview and receive rubric-based feedback.
- User should be able to build a resume by speaking in their native language and export it as a PDF.

Non-Functional Requirements

- Good response time
- Security (password hashing, JWT auth, token blacklisting)
- Scalability (SQLite for dev, PostgreSQL for production)
- User-friendly interface
- Robust error handling and graceful LLM fallbacks

5. System Architecture / Workflow

This section explains how the complete system works end-to-end.

Architecture Overview

The system follows a layered architecture:

- React 19 + TypeScript UI (Vite, AppShell, Web Speech API, jsPDF) — presentation layer.
- FastAPI REST API layer (CORS, Auth, Validation, Routers) — application layer.
- Core Logic Services (resume_parser, candidate_preprocessor, internship_index, llm_ranker, cover_letter_service, rag/, interview_agent_service, voice_resume_service) — business logic.
- Storage & Vectors (SQLite/PostgreSQL ORM, FAISS vector index with all-MiniLM-L6-v2 embeddings, local uploads directory) — persistence layer.
- External AI Models (Groq API — openai/gpt-oss-120b & llama-3.3-70b-versatile; Sentence-Transformers for local vectorization) — AI/ML layer.

Request Flow Examples

Resume Upload Flow: User uploads PDF/DOCX → file magic bytes validated → text extracted → regex & taxonomy extract skills/metadata → saved to database → embeddings generated → top internship matches queried from FAISS → results returned to client.

Voice Resume Flow: User speaks (e.g. in Telugu) → browser Web Speech API captures speech in continuous mode → transcript sent to /voice-resume/extract → Groq LLM translates to fluent English, fixes grammar, and extracts structured fields → client displays live transcript & extracted profile → user clicks "Build Resume Now" → /voice-resume/generate formats an ATS resume in English → client exports the formatted PDF via jsPDF.

High-Level Data Flow

User → React Frontend → FastAPI → Core Logic Services → FAISS / Groq LLM → Response → User

For AI-driven matching specifically:

Resume → Text Extraction → Entity Extraction → Embedding (all-MiniLM-L6-v2) → FAISS Retrieval → Feature Reranking → LLM Explanation → Ranked Matches

6. Backend Service Breakdown

6.1 Authentication & User Session Management

Files: app/auth.py, app/dependencies.py, app/routers/auth.py

- Passwords are hashed using `passlib.context.CryptContext` with the `bcrypt` scheme.
- `create_access_token()` generates signed JWTs using `SECRET_KEY` and `HS256`; tokens are valid for 30 minutes.
- `get_current_user()` extracts Bearer tokens via `HTTPBearer()`, validates signature and expiration, and checks the `BlacklistedToken` table so logged-out tokens are immediately invalid.

6.2 Resume Ingestion & Extraction Engine

Files: app/parser.py, app/resume_parser.py, app/routers/resume.py

- File validation via magic-byte checks (`%PDF-` for PDF, `PK\x03\x04` for DOCX); max file size capped at 5MB.
- PDF text extracted via `pdfplumber.extract_text()`; DOCX extracted via `docx.Document()`.
- Regex patterns identify email addresses, phone numbers, and LinkedIn/GitHub URLs.
- Skill matching evaluates tokens against `TECHNICAL_SKILLS` and `SOFT_SKILLS` dictionaries.
- Section segmentation splits text into Work Experience, Projects, Education, and Certifications.

6.3 Multi-Stage Semantic Internship Matcher

Files: app/candidate_preprocessor.py, app/internship_index.py, app/llm_ranker.py

Stage 1 — Dense Vector Retrieval: Candidate profile text is embedded using `sentence-transformers/all-MiniLM-L6-v2` (384-dimensional) and matched against a pre-built FAISS index of internships, returning the top-K semantic candidates.

Stage 2 — Feature-Based Scoring & Filtering: Exact skill overlap, title relevance, and degree alignment are combined into a normalized match percentage.

Stage 3 — LLM Re-ranking & Explanation: The Groq LLM analyzes the top matches against the candidate resume, explains why each candidate fits, highlights missing prerequisites, and gives tailored tips.

6.4 AI Cover Letter Generator

Files: app/cover_letter_service.py, app/routers/cover_letter.py

- Combines parsed resume data with company name, job role, and job description to produce standard or dynamic-tone letters.
- Resilience: lazy Groq import — if `GROQ_API_KEY` is missing or the call fails, a deterministic template fallback populates candidate details into a standard letter.

6.5 RAG Career Knowledge Assistant

Files: app/rag/ingest.py, app/rag/retriever.py, app/rag/generator.py, app/routers/career_assistant.py

- Ingestion: Markdown & DOCX guides chunked into overlapping segments (500 tokens, 100 overlap).
- Retrieval: dense retrieval via FAISS based on the user's query.
- Synthesis: grounded LLM response generation with source citations.

6.6 Interview Preparation & Mock Interview Agent

Files: app/interview_agent_service.py, app/routers/interview_agent.py

- Role Recommendation: evaluates the user's parsed skills/projects to suggest 3–5 best-fit job roles.
- Technical & HR Questions: generates role-specific and behavioral questions mapped to the STAR method (Situation, Task, Action, Result).
- Personalized Roadmaps: 2–4 week step-by-step interview preparation schedules with concrete milestones.
- Project Defense: deep-dive questions on architectural decisions, performance challenges, and trade-offs in the candidate's own projects.
- Interactive Mock Interview: multi-turn conversation where the AI asks questions sequentially, scores answers 1–10 on Clarity, Technical Depth, and STAR Structure, and gives constructive feedback before advancing.

6.7 Voice Resume Architect & Multilingual Translation

Files: app/voice_resume_service.py, app/routers/voice_resume.py, frontend/src/utls/resumePdf.ts

- Accepts free-form spoken speech in Telugu, Hindi, Tamil, Spanish, French, English, and more; all output is translated to standard professional English, with names in regional scripts transliterated into Latin script.
- Automatically removes verbal fillers (um, uh, like, you know) and repetitive phrases, elevating spoken phrases into ATS-friendly statements with high-impact action verbs.
- Template auto-classification detects Technical vs Blue-Collar/Trades backgrounds and formats section headings accordingly.
- Live editable transcript allows manual review before generation; resumes can be generated even from sparse input.
- Client-side PDF generation (jsPDF) renders an executive-styled, multi-page ATS resume.

7. Database Schema & Entity Models

Defined in app/models.py.

Table: users

Column	Type	Constraints	Description
id	Integer	Primary Key, Auto-increment	Unique user ID
name	String(100)	Nullable	Full candidate name
email	String(100)	Unique, Index, Not Null	User login email
hashed_password	String(255)	Not Null	Bcrypt password hash
created_at	DateTime	Default utcnow	Account creation timestamp

Table: blacklisted_tokens

Column	Type	Constraints	Description
id	Integer	Primary Key, Auto-increment	Token record ID
token	String	Index, Not Null	Revoked JWT Bearer string
blacklisted_at	DateTime	Default utcnow	Logout timestamp

Table: resumes

Column	Type	Constraints	Description
id	Integer	Primary Key, Auto-increment	Resume ID
user_id	Integer	Foreign Key (users.id)	Owner user reference
filename	String(255)	Not Null	Uploaded file name
file_path	String(500)	Not Null	Storage path on disk
parsed_data	JSON	Nullable	Extracted structured resume fields
created_at	DateTime	Default utcnow	Upload timestamp

Table: cover_letters

Column	Type	Constraints	Description
id	Integer	Primary Key, Auto-increment	Cover letter ID
user_id	Integer	Foreign Key (users.id)	Owner user reference
company_name	String(100)	Not Null	Target company
job_role	String(100)	Not Null	Target job title
content	Text	Not Null	Generated letter body
created_at	DateTime	Default utcnw	Generation timestamp

8. Complete REST API Specification

Authentication (/auth)

- POST /auth/register — create user (username, password, name).
- POST /auth/token — OAuth2 password request → { access_token, token_type }.
- POST /auth/logout — blacklists the current JWT token in the database.
- GET /auth/me — returns current user details.

Resume & Matching (/resume)

- POST /resume/upload — multi-part file upload (.pdf, .docx); parses entities and stores the resume record.
- GET /resume/my-resumes — lists all resumes uploaded by the current user.
- GET /resume/latest — retrieves the most recent parsed resume data.
- POST /resume/recommend — runs dense FAISS retrieval and reranking for the latest uploaded resume.

Cover Letter (/cover-letter)

- POST /cover-letter/generate — input { company_name, job_role, job_description, resume_id? } → { cover_letter }.
- GET /cover-letter/history — fetches the user's previous cover letters.

Career Assistant RAG (/career-assistant)

- POST /career-assistant/chat — input { message, conversation_history? } → { reply, sources }.

Interview Agent (/interview-agent)

- POST /interview-agent/chat — full multi-turn endpoint supporting role recommendations, skills analysis, technical & HR question generation, and multi-turn mock interview evaluation.
- GET /interview-agent/history — fetches saved interview interaction history.

Voice Resume Architect (/voice-resume)

POST /voice-resume/extract — accepts a spoken transcript, conversation history, and language hint; returns extracted structured fields, missing fields, a follow-up question, completion status, and detected language.

POST /voice-resume/generate — accepts extracted_data and a template hint; returns the formatted resume text, the template used (technical / blue-collar), and the generation method (llm or fallback).

9. Frontend Architecture & UI Systems

App Layout & Routing

frontend/src/App.tsx uses React Router v7:

- Public routes: / (Landing), /login, /register.

Protected routes wrapped with `<ProtectedRoute>` and `<AppShell>`:

- /app — Overview dashboard
- /app/resume — Traditional Resume Upload & Parsing
- /app/voice-resume — Voice Resume Architect
- /app/interview-agent — Interview Preparation & Mock Sessions
- /app/cover-letter — Cover Letter Generator
- /app/internships — Internship Matching
- /app/profile — Profile Settings

Client Architecture Patterns

- API Client: frontend/src/api/client.ts provides a unified `request<T>()` fetch wrapper that automatically appends the `Authorization: Bearer <token>` header and parses backend errors.
- Theme System: responsive dark mode managed via CSS variables (`--accent`, `--surface`, `--text`, `--border`) using a premium obsidian/indigo palette.
- Audio & Speech Engine: Web Speech API (continuous recognition with a 3.5s auto-silence timer) for the Interview Agent mic; SpeechSynthesis for text-to-speech playback of agent replies; AudioContext/OfflineAudioContext for decoding audio tracks client-side in the Voice Resume flow.
- PDF Generation: frontend/src/utils/resumePdf.ts creates executive A4 PDFs with 40pt margins, bold headers, divider lines, and text wrapping.

10. Environment Configuration (.env)

Create a .env file in the root backend directory:

```
GROQ_API_KEY=gsk_your_groq_api_key_here
SECRET_KEY=your-secret-key-change-in-production
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=30
DATABASE_URL=sqlite:///./career_companion.db
# HF_TOKEN=hf_your_token_here (optional — avoids rate limits on embedding download)
```

11. Step-by-Step Setup Guide on a New Machine

Step 1: Clone or Copy Repository

```
git clone <repository_url>
cd "AI-Career-Companion-Agent"
```

Step 2: Set Up Backend Environment (Python 3.10+)

```
python -m venv venv
# Windows: .\venv\Scripts\Activate.ps1
# Linux/macOS: source venv/bin/activate
python -m pip install --upgrade pip
pip install -r requirements.txt
```

Step 3: Configure Environment Variables

```
echo GROQ_API_KEY=your_key_here > .env
echo SECRET_KEY=your-secret-key >> .env
```

Step 4: Build RAG Index

```
python -m app.rag.ingest
```

Step 5: Initialize Frontend Environment (Node.js 18+)

```
cd frontend
npm install
npm run build
```

Step 6: Start Servers

Terminal 1 (Backend):

```
uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

API docs available at: <http://127.0.0.1:8000/docs>

Terminal 2 (Frontend):

```
npm run dev
```

App available at: <http://localhost:5173>

12. Deployment Guide (Railway + Vercel)

Backend → Railway

- Push the repository to GitHub.
- Create a new Railway project and link the repository.
- Railway auto-detects Python via Nixpacks using railway.json.
- Set environment variables in the Railway dashboard: GROQ_API_KEY, SECRET_KEY, DATABASE_URL (use Railway's PostgreSQL plugin for production, or leave blank for SQLite).
- Deploy — Railway runs: `uvicorn app.main:app --host 0.0.0.0 --port $PORT`

Frontend → Vercel

- Import the repository into Vercel.
- Set Root Directory to frontend.
- Set Build Command to `npm run build` and Output Directory to `dist`.
- Set environment variable VITE_API_BASE_URL to the Railway backend URL (e.g. `https://your-app.railway.app`).
- `frontend/vercel.json` handles SPA routing — all paths rewrite to `/index.html`.

13. Testing

This section demonstrates that the implemented features work as intended.

Testing Types Performed

- Functional Testing
- API Testing (via /docs Swagger UI)
- Integration Testing (frontend ↔ backend)
- AI/RAG Testing (retrieval relevance, LLM fallback behavior)

Test Cases

[Add test case table: Test ID, Feature, Input, Expected Result, Actual Result, Status]

14. Challenges & Solutions

[Describe the real, specific problems faced during development and how each was resolved — e.g. handling missing/invalid GROQ_API_KEY gracefully via deterministic fallback templates, or multilingual transliteration accuracy for names in regional scripts.]

15. Final Project Status

Completed Features

-  Resume upload & structured parsing (PDF/DOCX)
-  Multi-stage semantic internship matching (FAISS + reranking + LLM explanation)
-  AI cover letter generation with fallback templates
-  RAG career knowledge assistant with source citations
-  Interactive interview preparation & multi-turn mock interview agent
-  Voice-first multilingual resume architect with ATS PDF export

Final Outcome

The project successfully implements resume parsing, semantic internship matching, cover letter generation, a RAG-based career advisor, and a full interview preparation/mock-interview agent, along with a multilingual voice resume architect. Core features have been tested end-to-end across the FastAPI backend and React frontend. Additional deployment hardening and UI refinements are planned for the next phase.

Appendix: Summary for LLM Agents & Re-implementers

When implementing or extending services in another instance of this project:

- Always inherit from schemas in `app/schemas.py` to keep request/response serialization consistent across the 7 routers.
- Follow lazy-import conventions for external AI clients (from `groq` import `Groq` inside functions with `try/except None` fallbacks) so tests and offline flows run smoothly.
- Use the `get_current_user` dependency on all sensitive endpoints to ensure per-user data isolation.
- All multilingual user speech must be translated to English at the extraction and generation stages for uniform ATS formatting and PDF rendering.